

■ ЛЕКЦИЯ 2 · ИНСТРУМЕНТЫ РАЗРАБОТКИ



Git: основы и устройство

Зачем нужна система контроля версий, из чего Git собран внутри
и что на самом деле происходит при каждом коммите

Чернов Евгений

Магистратура «Искусственный интеллект», НИУ ВШЭ

План занятия

1

Зачем VCS

проблема версий, коллаборация, откаты

2

Модель работы

рабочая директория, индекс, репозиторий

3

Объектная модель

blob, tree, commit, tag и SHA

4

История как граф

коммит = снимок + родитель, DAG

5

Ссылки

HEAD, ветки и указатели

6

Удалённые репозитории

clone, fetch, pull, push

Зачем нужна система контроля версий?

Без VCS история проекта живёт в именах файлов и в голове:

● ● ● проект без git

```
model.py
model_final.py
model_final_v2.py
model_final_v2_FIX.py
model_final РАБОЧАЯ.py
# кто, когда и зачем — неизвестно
```



История — кто, когда и почему менял каждую строку



Откаты — вернуться к любому прошлому состоянию



Коллаборация — несколько человек в одном проекте без хаоса



Ветвление — параллельные линии работы и эксперименты

Git — это не GitHub

Их постоянно путают. Git — это инструмент, GitHub — один из сервисов, где можно хранить git-репозитории.

git Инструмент

- программа на вашем компьютере
- версионирование кода локально
- работает без интернета
- один и тот же для всех сервисов

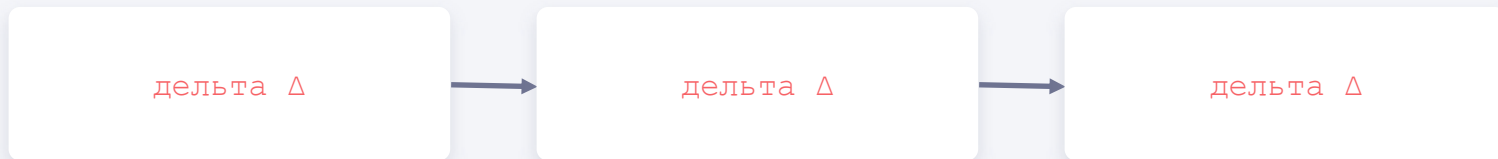
GitHub Сервис (хостинг)

- сайт, хранит репозитории в облаке
- добавляет Pull Request, Issues, CI
- нужен для совместной работы
- альтернативы: GitLab, Bitbucket

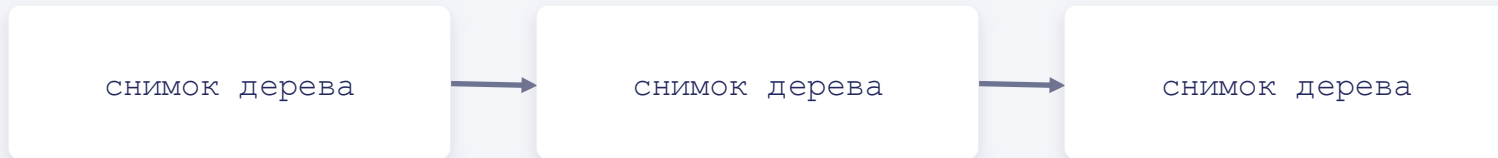
Git хранит снимки, а не изменения

Многие VCS хранят набор дельт (что изменилось). Git при каждом коммите сохраняет снимок всего дерева целиком, а неизменные файлы переиспользует по ссылке.

Другие VCS: цепочка изменений



Git: последовательность снимков (⚡ ключевое отличие)



Три состояния: рабочая директория → индекс → репозиторий



`git add` переносит изменения в индекс, `git commit` фиксирует индекс в историю. Индекс — это то, что отличает Git от «просто сохранить».

Первый цикл работы

● ● ● терминал

```
$ git init
Initialized empty Git repository
$ git add model.py
# файл ушёл в индекс (staging)
$ git commit -m "add model"
[main a1b2c3d] add model
$ git status
nothing to commit,
working tree clean
```

git init

создать репозиторий (появится папка .git)

git add

добавить изменения в индекс

git commit

зафиксировать снимок в историю

git status

посмотреть, что где находится

Настройка перед первым коммитом

Git подписывает каждый коммит вашим именем и почтой. Их нужно задать один раз — иначе автор будет пустым или чужим.

 терминал

```
$ git config --global \
    user.name "Ivan Petrov"
$ git config --global \
    user.email "ivan@edu.hse.ru"
$ git config --list
user.name=Ivan Petrov ...
```

--global

настройка для всех
ваших репозиториев

user.name

имя автора — попадёт
в каждый коммит

user.email

почта автора (лучше
совпадающая с GitHub)

--list

показать текущие
настройки

01

■ РАЗДЕЛ

Как Git устроен внутри

Объектная модель, хеши и граф истории — то, что снимает всю «магию»

Content-addressable storage: адрес = хеш содержимого

Git — это хранилище объектов, где имя каждого объекта — SHA-1 хеш его содержимого. Одинаковое содержимое → один и тот же хеш → хранится один раз.



как считается адрес объекта

```
$ echo 'hello' | git hash-object --stdin
```

```
ce013625030ba8dba906f756967f9e9ca394464a
```

```
# └─ 40-символьный SHA-1: и адрес, и контроль целостности
```



Меняете содержимое — меняется хеш. Поэтому историю нельзя незаметно подменить: любой правкой ломается вся цепочка адресов.

Четыре типа объектов

blob

содержимое файла
(без имени и пути)

tree

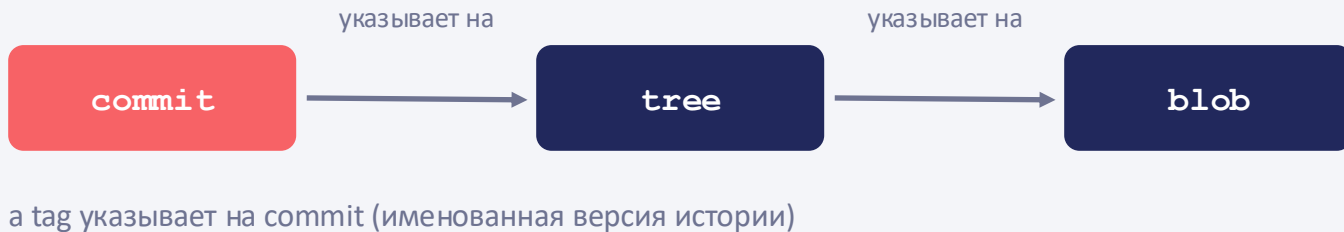
каталог: имена →
ссылки на blob/tree

commit

снимок дерева +
родитель + автор

tag

именованная ссылка
на коммит (версия)



Коммит изнутри: снимок + родитель

```
что лежит в объекте commit

$ git cat-file -p HEAD
tree 9d2f1a...
parent 7c4e88...
author Ivan <ivan@hse.ru> 172...
committer Ivan <ivan@hse.ru> 172...
```

add baseline model

tree

снимок всего дерева файлов на момент коммита

parent

ссылка на предыдущий коммит (у первого — нет)

author

кто и когда написал изменения

message

зачем это изменение — человекочитаемо

История — это направленный граф (DAG)

Каждый коммит ссылается на родителя. Стрелки идут в прошлое: история — не линия, а граф. Слияния создают коммит с двумя родителями.

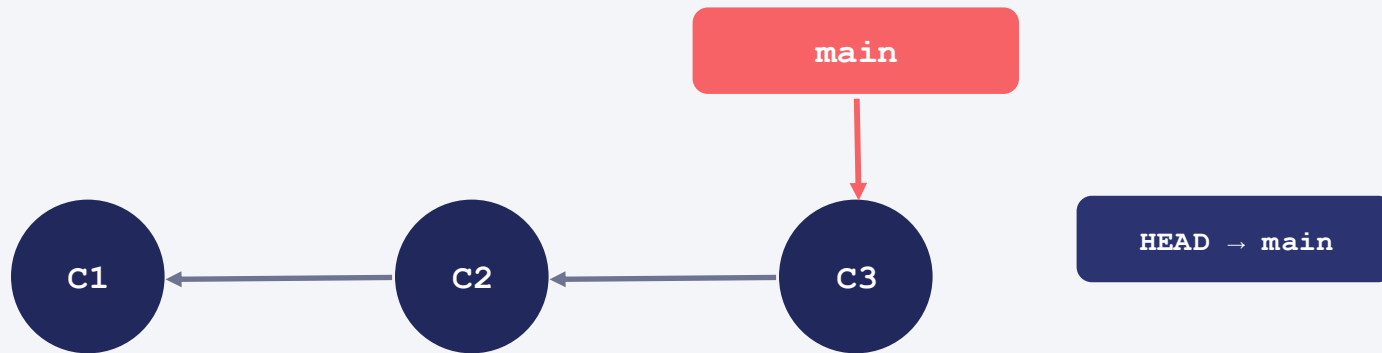


стрелки → в прошлое (к родителю)

merge-коммит M имеет двух родителей: C3 и F

Ветка — это просто указатель на коммит

Ветка (branch) — это лёгкий, перемещаемый указатель на коммит. HEAD указывает, где вы сейчас. Создать ветку = записать 41 байт, поэтому ветвление в Git дешёвое.



новый коммит просто сдвигает указатель main вперёд

Что лежит внутри .git/

структура репозитория

```
.git/  
├── HEAD  
├── config  
├── objects/  
│   └── ce/013625...  
├── refs/  
│   └── heads/main  
└── index
```

HEAD

куда указывает текущая позиция

objects/

все blob / tree / commit по хешам

refs/heads/

ветки — файлы с хешом коммита

index

тот самый индекс (staging)

config

настройки репозитория

Команды, которыми смотрят и сравнивают

`git status`

что в рабочей директории и индексе

`git log --oneline`

история коммитов кратко

`git diff`

несохранённые изменения (WD vs индекс)

`git diff --staged`

что уйдёт в коммит (индекс vs HEAD)

`git show <commit>`

содержимое конкретного коммита

`git cat-file -p <sha>`

заглянуть в сырой объект

.gitignore — что НЕ версионировать

Не всё стоит хранить в git. Файл .gitignore перечисляет пути, которые Git игнорирует: кэши, секреты, большие данные, артефакты сборки.

● ● ● пример для Python-проекта

```
# .gitignore
__pycache__/  
*.pyc  
.venv/  
.env  
data/*.csv
```

Секреты

.env, ключи, пароли — никогда в git

Кэши / артефакты

__pycache__, *.pyc, build/, dist/

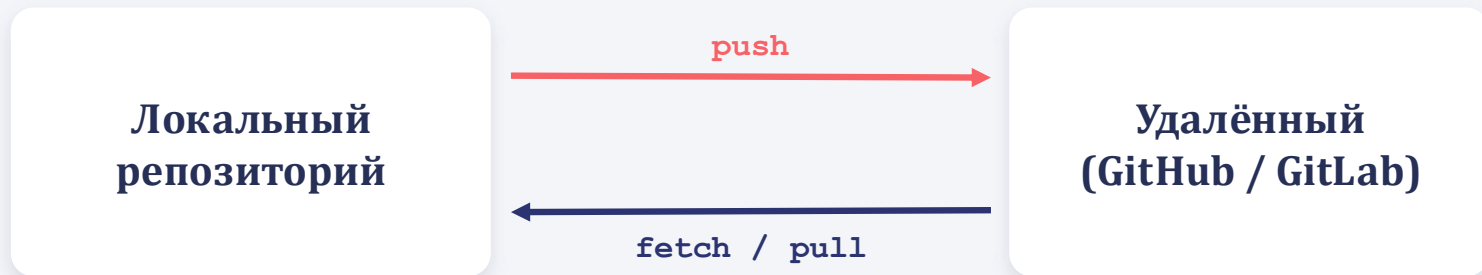
Окружения

.venv/, node_modules/ — ставятся заново

Данные

большие датасеты, веса моделей

Удалённые репозитории: локально ↔ сервер



clone — скопировать удалённый репозиторий к себе

fetch — забрать чужие изменения, не трогая свои

pull — fetch + слияние в текущую ветку

push — отправить свои коммиты на сервер

Что важно унести

1

Git хранит снимки

каждый коммит — снимок всего дерева,
а не дельта

2

Всё адресуется хешом

SHA содержимого = имя объекта и
защита целостности

3

Три состояния

рабочая директория → индекс →
репозиторий

4

История — это граф

коммиты связаны через parent, ветка —
указатель

Дальше — **Лекция 3: ветвление и воркфлоу**. Практика — ДЗ-2 через Pull Request.